

23.2_Assisgnment_1275

AI Assisted Coding

Name : D.Charanvitha

H.no : 2303A51408

Batch : 07

- Students choose a real-world problem to solve with an AI- assisted application (e.g., e-commerce billing system, health monitoring dashboard, student attendance tracker).

- Prepare a project proposal including:

o Objective o Features o

Tools/technologies to be used o

Expected output

```
Task 1 – Project Proposal

[1]
✓ 0s
# project_proposal.py

proposal = {
    "title": "AI-Assisted Student Attendance Tracker",
    "objective": "Automatically track and manage student attendance using AI features.",

    "Features": [
        "Mark attendance",
        "Store student records",
        "Generate reports",
        "AI-based attendance prediction"
    ],

    "technologies": [
        "Python",
        "Flask (Backend)",
        "SQLite (Database)",
        "HTML/CSS/JS (Frontend)"
    ],

    "expected_output": "A web app that records attendance and predicts future attendance."
}

for key, value in proposal.items():
    print(f"{key.upper()}: \n{value}\n")
```

```
[1] ✓ Os [play] "HTML/CSS/JS (Frontend)"
],
"expected_output": "A web app that records attendance and predicts future attendance."
}

for key, value in proposal.items():
    print(f"{key.upper()}: \n{value}\n")

... TITLE:
AI-Assisted Student Attendance Tracker

OBJECTIVE:
Automatically track and manage student attendance using AI features.

FEATURES:
['Mark attendance', 'Store student records', 'Generate reports', 'AI-based attendance prediction']

TECHNOLOGIES:
['Python', 'Flask (Backend)', 'SQLite (Database)', 'HTML/CSS/JS (Frontend)']

EXPECTED_OUTPUT:
A web app that records attendance and predicts future attendance.
```

Task 2 – Application Design

- Use AI tools to assist in:
 - o Designing system architecture diagrams.
 - o Defining database schema (if needed).
 - o Creating wireframes/UI mockups for frontend design.

```
Task 2 – Application Design

[2] ✓ Os [play] # architecture.py

architecture = """
[ User Interface ]
    ↓
[ Flask Backend API ]
    ↓
[ SQLite Database ]
    ↓
[ AI Model (Prediction) ]
"""

print("System Architecture:\n", architecture)

... System Architecture:

[ User Interface ]
    ↓
[ Flask Backend API ]
    ↓
[ SQLite Database ]
    ↓
[ AI Model (Prediction) ]
```

```
[ ]
# database.py

import sqlite3

conn = sqlite3.connect("attendance.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    attendance INTEGER
)
""")

conn.commit()
conn.close()

print("Database created successfully!")
```

```
[3]
✓ Os
▶
html_code =
<!DOCTYPE html>
<html>
<head>
    <title>Attendance Tracker</title>
</head>
<body>
    <h1>Mark Attendance</h1>
    <form method="POST" action="/mark">
        Name: <input type="text" name="name"><br>
        Attendance: <input type="number" name="attendance"><br>
        <button type="submit">Submit</button>
    </form>
</body>
</html>
"""

print(html_code)

▼
...
<!DOCTYPE html>
<html>
<head>
    <title>Attendance Tracker</title>
</head>
<body>
    <h1>Mark Attendance</h1>
    <form method="POST" action="/mark">
        Name: <input type="text" name="name"><br>
        Attendance: <input type="number" name="attendance"><br>
        <button type="submit">Submit</button>
```

Task 3 – AI-Assisted Code Development

- Implement the application in phases:
 - o Frontend: Web/mobile UI with HTML/CSS/JS (or framework).
 - o Backend: RESTful APIs or server logic.
 - o Database: Schema design, SQL queries.
 - o Integration: API calls, authentication, error handling.

Task 3 – AI-Assisted Code Development

```
[5]
✓ 2s # ai_model.py

from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data: days vs attendance
days = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
attendance = np.array([80, 82, 78, 85, 87])

model = LinearRegression()
model.fit(days, attendance)

# Predict future attendance
future_day = np.array([[6]])
prediction = model.predict(future_day)

print("Predicted Attendance:", prediction[0])

... Predicted Attendance: 87.50000000000001
```

Task 4 – Documentation & Code Quality

- Generate automatic documentation and inline comments using AI.
- Ensure clear README.md for project setup and usage.
- Conduct AI-assisted code review to refine readability and maintainability.

```
[6]
✓ 0s # readme_generator.py

readme = """
# AI Attendance Tracker

## Setup
1. Install dependencies:
   pip install flask sklearn

2. Run the app:
   python app.py

## Features
- Add students
- View attendance
- Predict attendance

## Author
Student Project
"""

with open("README.md", "w") as file:
    file.write(readme)

print("README.md created!")

... README.md created!
```

Task 5 – Testing & Debugging

- Use AI to:
 - o Generate unit tests.
 - o Identify vulnerabilities/security flaws.
 - o Suggest optimizations for performance.

Task 6 – Deployment & Presentation

- Deploy application on a cloud/free hosting platform (Heroku, GitHub Pages, Firebase, etc.).
 - Prepare a final presentation including:
 - o Problem statement
 - o Solution architecture
 - o Live demo
 - o Reflection on AI assistance and limitations
- Expected Outcomes:
- A complete, deployable software application.
 - Students gain experience in end-to-end development with AI tools.
 - Improved skills in teamwork, documentation, and ethical AI usage.



The screenshot shows a code editor window titled "Task 6 – Deployment (Basic Script)". The editor contains a Python script named `# deployment.py`. The script imports the `os` module and prints a list of steps to deploy a Flask application. The output of the script is displayed in a terminal window below the code editor.

```
[8]
✓ Os
# deployment.py

import os

print("Steps to Deploy:")
print("1. Push code to GitHub")
print("2. Connect to Render / Heroku")
print("3. Deploy Flask app")
print("4. Access live URL")

... Steps to Deploy:
1. Push code to GitHub
2. Connect to Render / Heroku
3. Deploy Flask app
4. Access live URL
```

